

BIBLIA

Modern AI use, subagents, SDD, AGENTS.md, GitHub, prompt engineering, and harness engineering.

Authorship

Nicolás Ezequiel Melluso

nicolas.e.melluso@gmail.com

linkedin.com/in/nicolas-ezequiel-melluso

github.com/Nicolas-Melluso

General Contents

01	Modern Use of Artificial Intelligence	How to move from chatting with a model to working with a system of thinking, execution, and verification
02	Intelligent Use of Subagents	Criteria, patterns, and operational closure for better delegation
03	SDD and Support Structure	Specification-Driven Development applied to AI-enabled repositories
04	AGENTS.md, .github and Prompt-Style Commands	How to build a repository prepared for agents, SDD, Copilot, workflows, and reusable prompts
05	Prompt Engineering and Harness Engineering	From loose prompts to versioned, evaluable, and production-grade systems

VOLUME 01

Modern Use of Artificial Intelligence

How to move from chatting with a model to working with a system of thinking, execution, and verification

The core idea

Using artificial intelligence in a modern way is not opening a chat, asking "do this," and accepting the first response. That was the first stage. The modern stage is treating AI as a work layer: a combination of assistant, technical peer, researcher, executor, reviewer, and memory system. The difference is not in writing longer prompts. It is in designing a flow where every interaction leaves reusable context, artifacts, tests, and decisions.

The immature way to use AI is conversational and disposable: you ask something, get a response, copy one part, and move on. The mature way is operational: you define the objective, provide context, make constraints explicit, split the work, verify the result, and store what was learned. Value appears when AI stops being a text machine and starts functioning as an extension of the development, research, or production process.

This volume proposes a simple working model: think of AI as a system with four layers.

1. Context layer: what AI must know before acting.
2. Task layer: what it must produce now.
3. Verification layer: how to know the result is useful.
4. Memory layer: where it is recorded to avoid repeating the same reasoning.

When those four layers exist, AI can help with serious work: writing specifications, reviewing code, comparing alternatives, generating documentation, running tests, detecting risks, preparing presentations, training a person, or accelerating technical decisions. When they are missing, AI becomes brilliant at times and silently dangerous.

What changed in practice

The important change is not only that models are better. The change is that now you can work with agents connected to tools: editor, terminal, browser, repository, issue tracker, documentation, local databases, tests, linters, emulators, and pipelines. Before, the model answered from outside the work. Now it can participate inside the work.

This forces a change in how requests are made. A modern request does not just say "explain X." It says: "read these files, identify current behavior, propose a scoped change, implement it, run these tests, and leave me a summary with risks." AI stops being an oracle and becomes an operator under contract.

The human role also changes. Humans no longer win by typing everything manually. They win by defining good context, good constraints, good acceptance criteria, and good verification mechanisms. AI can produce a lot, but by itself it does not know which tradeoff is best for your business, which legal risk to accept, which technical debt to tolerate, or which user experience you want to defend.

The modern question is not "which prompt should I use." The modern question is "which work system makes results better every week."

The recommended work cycle

A robust AI flow can look like this:

```
Intent -> Context -> Plan -> Execution -> Verification -> Record -> Next iteration
```

Intent defines the desired outcome. Context reduces ambiguity. Plan avoids impulsive work. Execution produces concrete artifacts. Verification separates what is convincing from what is correct. Record prevents knowledge from being lost in chat. The next iteration turns work into accumulated learning.

A simple example:

```
Intent:
  I want to add magic-link authentication.

Context:
  Node/TypeScript repo, PostgreSQL, service-based architecture, tests with Vitest.

Plan:
  1. Locate current auth.
  2. Add token table.
  3. Implement service.
  4. Add unit and integration tests.
  5. Document environment variables.

Verification:
  npm test
  npm run typecheck
  manual test of login -> link -> session flow

Record:
  Short ADR about why magic link and not password.
  Spec of expected behavior.
  Rollback checklist.
```

That flow does not depend on a specific tool. It works with Codex, Copilot, Cursor, Claude Code, Gemini CLI, or a custom agent. Maturity is in the process.

The minimum unit of context

AI works better when it receives packaged context, not a cloud of information. The minimum unit of context for a serious task should include:

Element	What it is for	Example
Objective	Prevents optimization of the wrong thing	"Reduce abandoned checkout errors"
Current state	Provides a starting point	"Stripe return goes back to <code>/checkout/success</code> "
Constraints	Narrows solutions	"Do not touch prices or migrate provider"
Relevant files	Reduces blind exploration	<code>src/server.js</code> , <code>public/app.js</code>
Acceptance criteria	Defines completion	"If it returns from payment, show purchased state"
Verification	Forces testing	"Local smoke test and unit test"
Risks	Makes sensitive areas visible	"Do not leak secrets in logs"

Without that minimum unit, the model fills gaps with assumptions. Sometimes it gets it right. In real systems, sometimes it breaks things by following logic that seemed reasonable but did not belong to the project.

Good tasks for AI

AI is especially strong when it can operate on explicit information and when the result can be verified. Some high-value uses:

1. Summarize and map existing code.
2. Turn an idea into a reviewable specification.
3. Generate test cases from acceptance criteria.
4. Detect inconsistencies between docs, code, and tests.
5. Refactor a scoped area with a test suite.
6. Prepare migration or validation scripts.
7. Create runbooks for repeatable operations.
8. Review PRs with concrete rules.
9. Turn conversations into actionable tasks.
10. Create training material for another person.

AI is less reliable when asked to decide without data, invent business policies, touch many areas of the system at once, change infrastructure without permissions, or produce "final" content without human review. This does not mean it cannot help. It means it needs a stronger control frame.

The modern prompt

A modern prompt has the shape of an operational brief. It does not need to be poetic or huge. It needs to remove ambiguity.

Objective:

I want you to turn this idea into an implementation-ready SDD specification.

Context:

The product is a B2B app to manage claims. The repo uses Node.js, TypeScript, PostgreSQL, and GitHub Actions. We want to keep the scope small.

Input:

Idea: "allow an operator to reassign a claim to another team."

Constraints:

- Do not design a full screen yet.
- Do not assume new roles unless necessary.
- Separate business rules from UI.
- Include risks and open questions.

Output:

1. Summary of the problem.
2. Functional requirements.
3. Non-functional requirements.
4. Acceptance criteria.
5. Edge cases.
6. Implementation plan in 3 slices.
7. Recommended tests.

Quality:

If information is missing, mark it as an open question. Do not invent business data.

That format does three things: it provides direction, sets limits, and defines how to evaluate the response. It does not try to "inspire" the model. It aims to hire it for a task.

From chat to repository

The most important leap is moving knowledge from chat into the repository. Chat is fragile: it gets lost, contradicts itself, versions poorly, is not reviewed in PR, and does not run in CI. The repository, on the other hand, can store instructions, specs, decisions, prompts, tests, and workflows.

A modern organization usually separates:

Artifact	Audience	Function
<code>README.md</code>	New humans	Present the project and how to start
<code>AGENTS.md</code>	Code agents	Operational rules, commands, style, and verification
<code>.github/copilot-instructions.md</code>	Copilot	General instructions for responses in the repo
<code>.github/instructions/*.instructions.md</code>	Copilot by path	Specific rules by area of the code
<code>.github/prompts/*.prompt.md</code>	Humans and assistants	Reusable prompt-style commands
<code>.github/orquestador/sdd/*</code>	Team and agents	Specs, decisions, tasks, traceability
<code>.github/workflows/*</code>	CI/CD	Executable automation

The practical rule: what repeats should live as a file. If every time you ask for help you must explain the same commands, the same style, the same constraints, and the same testing criteria, that is not prompt engineering. It is context debt.

AI roles in a small team

Even if a single tool looks like "an assistant," it is useful to think in roles:

Role	What it does	Good output
Researcher	Reads, compares, summarizes, finds patterns	File map, risks, questions
Planner	Splits the work	Plan by slices, dependencies, verification
Implementer	Changes files	Scoped patch, tests, summary
Reviewer	Looks for errors	Findings with file and line
Documenter	Turns work into knowledge	README, ADR, runbook
Evaluator	Tests behavior	Report of commands and results

Subagents formalize that separation, but the mental model is useful even with a single chat. When one conversation tries to do everything at the same time, it becomes confusing. When each role has a concrete output, work becomes controllable.

Verification is not optional

AI can produce convincing text and code that looks correct. The only serious defense is verification. Verification can be automated or human, but it must exist.

Verification examples:

1. Unit and integration tests.
2. Typecheck, lint, and build.
3. Documented manual smoke test.
4. Comparison against acceptance criteria.
5. Review diffs file by file.
6. Test with real data or fixtures.
7. Evaluation with golden cases for prompts.
8. Security and permissions checklist.

The phrase "looks good" is not enough. A modern flow asks AI to say what it executed, what it could not execute, what it changed, what is missing, and what risk remains.

Useful memory, not infinite memory

Memory is useful when it reduces repetition and improves consistency. It is not useful when it turns into a dump of long notes. Useful memory has three properties:

1. It is retrievable: it is in a known path.
2. It is actionable: it contains decisions, commands, conventions, or learned errors.
3. It is verifiable: it does not replace the current repo state when that state may have changed.

Examples of good memory:

- The repo uses ``.github/orquestador`` as the context folder.
- Workflows are the only executable layer; the catalog only documents.
- Before closing runtime changes run ``npm test`` and ``npm run build``.
- On Windows, verify locks before renaming folders.

Examples of bad memory:

- The project is important.
- Sometimes it fails.
- Use good practices.

Modern memory does not store feelings. It stores operations.

30-day adoption plan

Week 1: organize context

Create `AGENTS.md`, document real commands, list constraints, and define where the SDD lives. The objective is not to cover everything. It is to let an agent enter the repo without losing half an hour guessing.

Deliverables:

1. Initial `AGENTS.md`.
2. Updated `README.md`.
3. `.github/orquestador/context/product.md`.
4. `.github/orquestador/context/architecture.md`.
5. Verification command list.

Week 2: work by specifications

Choose a small feature and write a spec before implementation. Include acceptance criteria, edge cases, non-goals, and tests. Then ask AI to implement only one slice.

Deliverables:

1. First SDD spec.
2. Plan by slices.
3. ADR if there is a relevant technical decision.
4. Associated tests.

Week 3: introduce reusable prompts

Create prompts for repeated tasks: planning a feature, reviewing a PR, generating tests, writing an ADR, preparing a runbook. Save them in `.github/prompts` or in the chosen orchestration folder.

Deliverables:

1. `plan-feature.prompt.md`.
2. `review-pr.prompt.md`.
3. `write-adr.prompt.md`.
4. `generate-tests.prompt.md`.

Week 4: measure quality

Add simple harnesses or evaluations. There is no need to set up a huge platform. Start with fixtures, expected cases, and a script that compares outputs.

Deliverables:

1. `evals/` folder.
2. Input fixtures.
3. Evaluation rubric.

4. Local script or validation workflow.

Common anti-patterns

Anti-pattern	Symptom	Fix
Giant prompt for everything	The model ignores parts	Split into persistent instructions and concrete tasks
No verification	Nice-looking but fabricated outputs	Define commands and acceptance criteria
Context only in chat	Everything repeats each session	Move rules into the repo
Agent with broad permissions	Risk of destructive changes	Ownership and minimum permissions
Everything in one subagent	False parallelism	Separate exploration, implementation, and review
Docs that do not run	Good intentions with no effect	Connect docs to workflows and checklists

Checklist for using AI in a modern way

Before asking:

1. I am clear about the final result.
2. I can name the relevant files or domains.
3. I know what I do not want it to touch.
4. I have a way to verify.
5. I can accept a first slice instead of the whole system.

During the work:

1. I ask for short plans for risky tasks.
2. I split research, editing, and review.
3. I keep file ownership.
4. I read diffs before closing.
5. I record new decisions.

At close:

1. I know what changed.
2. I know which tests passed.
3. I know which tests were not run.
4. I know which risks remain.
5. Reusable knowledge ended up in files.

Closing

Modern AI does not replace craft. It amplifies it when craft is organized. The user who gets the most value is not the one who knows the secret prompt trick, but the one who knows how to turn ambiguous work into verifiable units.

The final rule is simple: if the output matters, treat AI as part of a production system. Give it context, limits, tools, tests, and memory. Everything else is just chat.

VOLUME 02

Intelligent Use of Subagents

Criteria, patterns, and operational closure for better delegation

What this volume is for

Working with modern agents and subagents is not about distributing tasks at random. The difference between useful delegation and wasted time usually comes down to three things: the size of the problem, the quality of framing, and the level of control over the outcome. When done well, work accelerates, context is used better, and the main thread stays free for decisions that truly require human or architectural judgment.

This document proposes a practical way to use subagents in technical projects. It does not try to sell magic or full automation. The goal is more modest and more useful: to help you split work with judgment, ask each subagent for exactly what is needed, keep traceability, and close each step with evidence.

The core idea is this: a subagent is not a replacement for thinking. It is a bounded execution unit that works with minimal context, clear ownership, and a verifiable output. If any of those pieces is missing, it is better not to delegate yet.

When to delegate

Not everything deserves a subagent. Delegating too early creates noise, duplication, and answers that look correct but do not solve the problem. Delegating too late leaves you doing repetitive tasks manually that could have been solved in parallel.

A good practical rule is to delegate when several of these conditions are met:

- The task is well bounded.
- The expected output can be verified.
- The task does not require cross-cutting decisions with other parts of the system.
- There is enough repetitive, exploratory, or mechanical work to justify coordination cost.
- The required context fits in a few instructions and specific files.
- The risk of the subagent touching something sensitive is contained by ownership or read-only scope.

Good delegation examples:

- Finding where behavior is implemented.
- Mapping data flow between modules.
- Identifying existing tests for an area of the code.
- Drafting a document from notes or a previous structure.
- Running a targeted validation on a limited folder.
- Testing technical hypotheses that do not require editing many pieces at once.

Bad delegation examples:

- "Fix the whole system."
- "Redo the architecture."
- "Jump into the repo and improve whatever you see."
- "Make product decisions without acceptance criteria."

- “Optimize performance” without a baseline, a metric, and a scope.

If you cannot say what file or what output you want, you are probably not ready to delegate.

Explorer and worker

The most useful separation in a subagent workflow is **explorer** and **worker**.

Explorer

The explorer is for understanding. Its job is to read, map, compare, and return condensed information. It should not make changes unless the task is explicitly exploratory with local annotation, and even then it is best to keep it read-only by default.

Typical uses:

- Finding implementations.
- Following references.
- Summarizing existing patterns.
- Detecting relevant tests, scripts, or configurations.
- Comparing alternatives without touching code.

What you ask from it:

- cite specific files;
- summarize with short bullets;
- avoid proposing unnecessary solutions;
- identify uncertainties;
- state what it could not confirm.

The explorer is especially valuable when you still do not know the real change surface. Before editing, understand the map first.

Worker

The worker is for doing. It receives a bounded objective, a clear editing area, and verifiable output criteria. It can edit files, run commands, or prepare a patch, but always within explicit ownership.

Typical uses:

- Implementing a specific function.
- Creating a validation script.
- Adjusting a documentation file.
- Testing a hypothesis in a working branch or on a subset of files.
- Preparing fixtures or test data.

What you ask from it:

- touch only authorized files;
- do not revert other people's changes;
- explain what changed;
- validate with concrete commands;
- leave work ready for review.

The general rule is simple: explorer to reduce uncertainty, worker to execute an already-understood task.

Minimal context

One of the most common delegation mistakes is passing too much context. Giving a subagent "everything there is" seems safe, but usually degrades quality. More context means more noise, more cost, and more chances the agent will mix irrelevant signals.

Good minimal context answers these questions:

1. What you want to achieve.
2. What part of the system it can touch.
3. What files or paths are relevant.
4. What it must not touch.
5. How you know it finished well.

A useful framing can look like this:

- Objective: one concrete sentence.
- Scope: files and folders.
- Constraints: do not touch X, do not modify Y, do not change behavior outside Z.
- Expected output: summary, patch, findings list, or script.
- Verification: tests, commands, or manual review.

You do not need to explain the entire project history. You need to explain the portion the subagent needs to act without improvising.

What to include

- The exact problem.
- The output format.
- Files that define ownership.
- Validation commands.
- Acceptance criteria.

What to avoid

- Long stories with no operational relevance.
- Contradictory opinions.

- Old clues that no longer apply.
- Mental snapshots the agent cannot verify.
- Open instructions like “improve it a lot.”

If the subagent needs a key clarification to avoid mistakes, it is better to pause and reframe. If it only needs noisy details, do not pass them.

File ownership

Ownership is what prevents multiple agents from stepping on the same ground. Every task should have a clear boundary: which files a worker may edit, which files it may only read, and which parts of the system are off limits.

This is not just hygiene. It is a way to preserve integrity while working in parallel.

A good assignment includes:

- authorized file or folder;
- permission type: read, edit, or inspection-only;
- impact limits;
- criteria for considering a change invasive;
- confirmation that it must not revert other people’s work.

Example of good ownership:

- Worker A: `src/docs/intro.md` and `src/docs/glosario.md`, content edits only.
- Worker B: `scripts/validate-docs.mjs`, only this file and its associated test.
- Explorer C: any file under `src/`, but no editing.

Example of bad ownership:

- “Edit whatever is needed.”
- “Tidy up the whole module.”
- “See if you can find something better.”

When ownership is clear, review quality improves too. You know what changed, why it changed, and what remains out of scope.

Parallelism

Subagents shine when you can split independent work. Parallelism does not mean doing more things at the same time out of anxiety, but separating tasks that do not block each other.

There are three useful levels:

Exploration parallelism

Several explorers search for different information in parallel.

Example:

- one locates the implementation;
- another identifies tests;
- another summarizes similar patterns;
- another searches for risks or dependencies.

This helps a lot at the start of a large task. Instead of reading everything sequentially, you get a faster map.

Execution parallelism

Several workers make changes in different areas, as long as files do not overlap.

Example:

- one edits documentation;
- another adjusts a validation;
- another prepares examples or fixtures.

This requires discipline. If two workers share files, the parallelism assumption breaks and you start competing with merges or rewrites.

Verification parallelism

One subagent implements and another verifies from the outside.

Example:

- worker A changes a function;
- worker B checks that relevant tests exist and that the change does not break conventions;
- the main thread integrates the result.

This pattern is useful to separate production from evidence. Independent verification reduces confirmation bias.

When not to parallelize

It is not advisable to parallelize when:

- one task decision depends on another task outcome;
- the same file will be edited by multiple agents;
- architecture is still under discussion;
- coordination cost exceeds the savings;
- one mistake could contaminate several pieces at once.

Parallelizing is not a goal in itself. It is a tool when independence truly exists.

Anti-patterns

Anti-patterns often look like productivity at first and like operational debt later.

1. Vague delegation

You ask for something too broad and get something too generic. The agent fills gaps with assumptions.

Typical sign: the response sounds neat but does not land on concrete files or decisions.

2. Context dumping

You pass the whole repo, the whole chat, all notes. The agent loses focus.

Typical sign: long responses with relevant fragments mixed with noise.

3. Diffuse ownership

Nobody knows who can touch what. Cross-edits, collisions, and confusion appear.

Typical sign: "I thought that folder was free."

4. Subagents solving design

The subagent starts inventing strategy when it only had to execute one concrete slice.

Typical sign: it proposes restructuring everything before finishing the requested change.

5. False parallelism

Several tasks are launched "in parallel" but actually compete for the same area.

Typical sign: file conflicts, inconsistent results, or needing to redo work.

6. Closure without evidence

The subagent says it finished, but does not show how to validate it.

Typical sign: no command, no file, no acceptance criteria.

7. Mixing reading and editing without control

An agent explores and also touches out-of-scope things "while at it."

Typical sign: unrequested collateral changes.

The golden rule is strict but simple: if a task cannot be reviewed clearly, it was probably delegated poorly.

Model and effort matrix

Not all subagents need the same model type or reasoning level. It helps to think in a practical matrix: task complexity by required effort.

The idea is not to memorize names, but to use a reasonable combination for the work.

Mechanical tasks

- Examples: file inspection, search, pattern extraction, simple validations, formatting.
- Suggested model: a lightweight or fast one.

- Effort: low.
- Objective: speed and low cost.

Synthesis tasks

- Examples: summarizing findings, comparing options, condensing documentation, mapping flow.
- Suggested model: lightweight or medium, depending on material density.
- Effort: low to medium.
- Objective: organize information without oversizing the response.

Bounded implementation tasks

- Examples: editing one file, creating a script, adjusting a test.
- Suggested model: medium.
- Effort: medium.
- Objective: good technical judgment without excessive cost.

Complex debugging tasks

- Examples: bugs with multiple causes, cross integration, intermittent failures.
- Suggested model: a stronger one.
- Effort: medium to high.
- Objective: more reasoning capacity, fewer shortcuts.

Final review tasks

- Examples: reviewing an important patch, detecting regressions, questioning assumptions.
- Suggested model: stronger or at least different from the one that implemented.
- Effort: medium to high.
- Objective: independence and critical perspective.

Practical rule

- If the task is repetitive, do not spend a heavy model.
- If the task depends on fine criteria or crosses several pieces, raise the level.
- If the result will decide an important delivery, add independent review.

Effort should not be "always high." It should follow risk and ambiguity.

Delegation prompt examples

The best subagent prompts do not look like open requests. They look like well-written tickets.

Implementation exploration

Objective: find where the autosave flow is implemented.
Role: explorer.
Scope: read-only on `src/` and `tests/`.
Deliverable: list of relevant files, brief flow summary, and risks.
Do not make changes.
If anything is unclear, mark the uncertainty.

Test mapping

Objective: identify existing tests for route validation.
Role: explorer.
Scope: search in `tests/`, `spec/`, and CI scripts.
Deliverable: simple table with file, purpose, and coverage.
Do not edit anything.

Bounded implementation

Objective: add a helper to normalize titles in `src/utils/title.js`.
Role: worker.
Exclusive ownership: only `src/utils/title.js`.
Constraints: do not touch other files, do not change public interfaces.
Deliverable: final code, brief explanation, and verification command.

Document or draft

Objective: write a technical draft on subagent usage.
Role: worker.
Ownership: only `src/02-subagentes-inteligentes.md`.
Style: clear, serious, actionable, in neutral Rioplatense Spanish.
Include criteria, anti-patterns, examples, and a closure checklist.
Do not generate extra files.

Independent verification

Objective: review the change and look for logical errors or scope creep.
Role: explorer.
Scope: read the patch and touched files.
Deliverable: concrete findings, open questions, and validation suggestions.
Do not modify files.

Notice that in all cases the prompt defines role, scope, deliverable, and limits. That reduces ambiguity and improves output quality.

Closure checklist

Delegation does not end when the subagent replies. It ends when the result is verified and the main thread can continue without dragging unresolved doubts.

Closure checklist to always use:

- The objective was met in one verifiable sentence.
- The subagent worked within authorized ownership.
- There were no out-of-scope changes.
- Touched files were identified.
- The output is reproducible or reviewable.
- If there were code changes, a validation command exists.
- If there were document changes, content matches the requested structure.
- Uncertainties were made explicit.
- There are no conflicts with parallel work.
- The final result was read by the main thread before considering it closed.

A stricter version for sensitive tasks:

- I reviewed the diff.
- I verified no external files were touched.
- I ran the relevant validation command.
- I confirmed there are no hidden assumptions.
- I logged what remains pending, if anything was left out.

Recommended process

A robust workflow for subagents usually looks like this:

1. Define the task and its output criteria.
2. Separate exploration from execution.
3. Assign explicit ownership.
4. Choose model level and effort based on ambiguity and risk.
5. Run tasks in parallel only if they do not overlap.
6. Consolidate results in the main thread.
7. Verify closure with evidence.

If the task is large, this flow can be repeated in layers: explorers first, then workers, then independent review.

Signs you are doing well

You are doing well when:

- the subagent responds with less text but more precision;
- each output mentions concrete files or commands;
- the main thread decides faster because it received better-filtered information;
- parallelism reduces time without increasing rework;
- errors are detected before integration.

You are doing poorly when:

- you need to reinterpret everything they returned;
- surprise changes appear;
- context grows out of control;
- verification happens only at the end and with surprises;
- each subagent needs the same base explained again.

Operational closure

Using subagents intelligently is not a matter of quantity, but of form. Delegation quality depends more on scoping, ownership, and verification than on the number of agents involved.

If you need exploration, use explorer. If you need execution, use worker. If you need several things at once, split work so they do not collide. And if you cannot define clear ownership, it is still not the right time to delegate.

The best practice is not "making AI work alone." It is building a work chain where each piece has a bounded role, a checkable output, and explicit closure. That is when subagents stop being a promise and become a truly useful tool.

VOLUME 03

SDD and Support Structure

Specification-Driven Development applied to AI-enabled repositories

SDD, or *Specification-Driven Development*, is a way of working where the specification is not a decorative document or an isolated note: it is the central piece of the development flow. Instead of starting with loose code, we start with a clear definition of what we want to build, why it exists, how it will be validated, and which decisions are recorded along the way.

In a modern repository, and even more when AI is involved, SDD organizes work in layers. First the problem is defined. Then that problem is turned into a verifiable specification. Next it is broken down into tasks. Only then is it implemented. And at the end, traceability is left behind: what changed, what was discarded, what was tested, and what is ready to operate.

This is especially useful when the team uses AI as a copilot, draft generator, or analysis assistant. AI can accelerate a lot, but it can also invent assumptions, mix priorities, or produce code that “works” without respecting context. SDD puts useful boundaries around that. AI does not guess the objective: it reads it. It does not improvise acceptance criteria: it follows them. It does not replace decisions: it documents them or proposes them for approval.

What SDD solves

SDD solves a very concrete problem: the gap between intention and implementation. In large repositories, that gap often grows silently. An issue says one thing, a PR solves another, the code ends up doing something else, and nobody knows which version of the idea was correct.

With SDD, that chain is explicit:

- **requirements** : what need exists and who cares about it.
- **specs** : how the expected behavior is described.
- **decisions** : which alternatives were evaluated and which was chosen.
- **tasks** : which concrete steps must be executed.
- **acceptance criteria** : how we know it is done correctly.
- **traces** : what evidence connects the spec with code and tests.
- **runbooks** : how to operate it or recover it when it fails.
- **ADRs** : architecture decisions with context and consequences.
- **evaluation notes** : results of validations, assisted testing, or comparative analysis.

The advantage is not only documentation. It also improves execution. When the structure is good, AI generates less garbage, the team discusses better, and changes are easier to review.

Core principle

The main SDD rule is simple: each piece must have a semantic owner.

- The requirement explains the problem.
- The spec defines behavior.
- The decision explains the trade-off.
- The task organizes work.
- Acceptance closes scope.

- The trace allows following the thread.
- The runbook prepares operations.

If everything is mixed into one file, the system becomes fragile. If it is too atomized without criteria, it does too. SDD seeks a balance: separate enough so each thing has its own function, but not so much that understanding a feature requires opening twenty unrelated files.

Recommended folder

The suggested convention for this volume is to concentrate SDD material around `.github/orquestador/sdd`. That folder works as a governance and execution core for specs, decisions, and AI-assisted work.

A possible structure is this:

```
.github/orquestador/sdd/  
  README.md  
  index.md  
  requirements/  
    000-template.md  
    <area>--<id>.md  
  specs/  
    000-template.md  
    <feature>--<id>.md  
  decisions/  
    000-template-adr.md  
    <adr>--<id>.md  
  tasks/  
    000-template.md  
    <issue>--<id>.md  
  acceptance/  
    000-template.md  
    <feature>--<id>.md  
  traces/  
    000-template.md  
    <feature>--<id>.md  
  runbooks/  
    000-template.md  
    <system>--<id>.md  
  evaluations/  
    000-template.md  
    <feature>--<id>.md  
  examples/  
    sample-issue.md  
    sample-spec.md
```

Not all folders need to exist from day one. What matters is that the architecture is designed to scale without losing readability.

What goes in each folder

README.md

It is the entry point. It should say what SDD is in this repository, what the folder's goal is, and how to use it. It should not contain the full theory, only a short guide to navigate the system.

Expected content:

- folder purpose;
- naming convention;
- recommended reading order;
- how to create a new spec or a new decision;
- relation to issues, PRs, and general documentation.

index.md

It is the navigation map. It lists active specs, relevant ADRs, the most recent evaluations, and critical runbooks. It can also include status: draft, in review, approved, implemented, obsolete.

Expected content:

- catalog of living artifacts;
- cross-links;
- status per document;
- last update date;
- owner or area.

requirements/

Requirements live here. They are not technical tickets yet. They are business, product, or operations needs, pain points, objectives, or constraints.

A well-written requirement answers:

- what problem exists;
- for whom;
- what happens if it is not solved;
- what constraints exist;
- what signals would indicate success.

Brief example:

```
# REQ-014: reduce errors in customer onboarding
```

Problem: forms currently allow incomplete data to be saved, and that creates rework.

Objective: block invalid onboarding before persistence.

Constraints: do not break the current editing flow.

Impact: less manual support and fewer errors in reports.

specs/

The spec describes behavior. It no longer talks only about the problem, but about the expected system. It has to be verifiable. A good spec clearly defines inputs, outputs, rules, states, errors, permissions, and edge scenarios.

Expected content:

- context;
- scope;
- main behavior;
- edge cases;
- alternative flows;
- dependencies;
- acceptance criteria;
- risks;
- explicit assumptions.

The spec should not write code, but it can include pseudo-rules, payload examples, state tables, or sequences.

decisions/

ADRs and any structural decision worth remembering live here. If the spec says *what* to do, the decision explains *why* that path was chosen and not another.

Expected content:

- context;
- options considered;
- decision made;
- consequences;
- trade-offs;
- date;
- author or team.

Decision example:

```
# ADR-007: validate on server and not only on client
```

```
Backend validation is chosen as the source of truth.
```

```
Reason: the client can become outdated and is not reliable as the only barrier.
```

```
Consequence: part of the logic is duplicated, but operational risk is reduced.
```

tasks/

Tasks break implementation into small, verifiable steps. They are the bridge between the spec and the code. A good task does not describe a vague aspiration, but a concrete action.

Expected content:

- task objective;
- dependencies;
- files or modules involved;
- definition of done;
- expected evidence.

A badly written task says: “build the feature.” A useful task says: “add validation for field X in endpoint Y and cover it with an integration test.”

acceptance/

This folder contains acceptance criteria. Its function is to close the contract between what was requested and what was delivered. It must be concrete enough that a reviewer or AI can check it without inventing interpretations.

Expected content:

- pass/fail conditions;
- nominal scenarios;
- expected errors;
- observable behavior;
- non-functional requirements when applicable.

traces/

Traces connect the requirement, spec, tasks, and final result. They help identify where each decision came from and what evidence supports it.

Expected content:

- link `REQ -> SPEC -> TASK -> PR -> TEST` ;
- coverage summary;
- references to commits, issues, or PRs;
- notes on what was left out.

runbooks/

Runbooks document how to operate or recover a flow when something fails. They are especially useful when a spec ends in a feature with real impact: payments, authentication, jobs, synchronizations, migrations, or automated processes.

Expected content:

- common symptoms;
- diagnostic steps;
- recovery steps;
- rollback when applicable;
- alert signals;
- contacts or dependencies.

evaluations/

Evaluation notes go here: human testing, AI testing, internal benchmarks, before/after comparisons, risk analysis, or quality reviews.

Expected content:

- what was evaluated;
- with what method;
- what result it gave;
- what defects appeared;
- what decision was made from it.

examples/

This helps reduce friction. Having templates and concrete examples greatly improves adoption, especially when the team is just starting with SDD.

Expected content:

- sample issue;
- sample spec;
- sample ADR;
- sample task;
- sample checklist.

Work cycle for an issue or feature

A healthy SDD flow does not start in the PR. It starts earlier, when changing direction is still cheap.

1. Open the requirement

First the problem is written. Poetry is not needed. Precision is. The requirement must explain why the initiative exists and what pain it solves.

2. Convert it into a spec

Then the expected behavior is defined. Here we answer:

- what goes in;
- what comes out;
- what rules apply;
- what errors are allowed;
- what is out of scope for this version.

3. Record decisions

If there are relevant alternatives, an ADR is recorded. This prevents someone from re-arguing an already made decision in two months as if nothing had happened.

4. Split into tasks

Implementation is divided into small tasks. Each task should be doable, reviewable, and testable independently.

5. Define acceptance

Before touching code, acceptance criteria must already be written. If they are written later, they usually adapt to the result and lose value.

6. Implement with traceability

Each change must keep a connection to the spec. That can be through references in the PR, notes in the task, or links in the trace. What matters is that the thread is not broken.

7. Evaluate

At the end, what happened is recorded: tests run, problems found, missing coverage, pending debt, and final decision.

An example of a full cycle:

1. **REQ-014** detects errors in customer onboarding.
2. **SPEC-014** defines validations, messages, and states.
3. **ADR-007** chooses backend validation.
4. **TASK-014.1** adds validations.
5. **TASK-014.2** covers integration tests.
6. **AC-014** defines 5 acceptance criteria.
7. **TRACE-014** links issue, PR, and tests.
8. **EVAL-014** summarizes the result and open points.

Short templates

Requirement

```
# REQ-XXX: short title
```

```
Problem:
```

```
Objective:
```

```
Impacted users:
```

```
Constraints:
```

```
Risk of not doing it:
```

Spec

```
# SPEC-XXX: short title
```

```
Context:
```

```
Scope:
```

```
Out of scope:
```

```
Expected behavior:
```

```
Edge cases:
```

```
Acceptance criteria:
```

ADR

```
# ADR-XXX: short decision
```

```
Context:
```

```
Options:
```

```
Decision:
```

```
Consequences:
```

Task

```
# TASK-XXX: concrete action
```

```
What needs to be done:
```

```
Likely files:
```

```
Dependencies:
```

```
How it is validated:
```


Acceptance

```
# AC-XXX: feature or spec

- Given ...
- When ...
- Then ...
```

Trace

```
# TRACE-XXX: short title

REQ:
SPEC:
TASKS:
PR:
TESTS:
NOTES:
```

Runbook

```
# RUN-XXX: system or flow

Symptom:
Diagnosis:
Recovery:
Rollback:
```

Evaluation notes

```
# EVAL-XXX: short title

What was tested:
Criteria:
Result:
Problems:
Decision:
```

How to use SDD with AI

AI changes the value of documentation. Before, a spec mostly served humans. Now it also serves as a context contract for assistants.

A good flow with AI is this:

- AI reads the requirement and summarizes the problem;
- AI proposes an initial spec;
- the human validates scope and priorities;
- AI breaks down tasks and suggests tests;
- the human approves or corrects decisions;
- AI helps implement and review traceability;
- the human verifies final acceptance.

This works better when the repository has stable and readable artifacts. If the spec is disorganized, AI will produce disorganized responses. If acceptance is vague, AI will fill gaps with assumptions. If there are no ADRs, decisions are lost and context restarts each time.

The key is to use AI to accelerate reasoning, not replace it. SDD makes AI work on an explicit base. Instead of “build me a feature,” you ask “using this spec and these acceptance criteria, break down implementation and flag risks.” That language shift greatly improves output quality.

Anti-patterns

Mixing everything into one file

When requirements, spec, task, and decision live together without structure, the document becomes impossible to use. Nobody knows which part is the source of truth.

Ambiguous specifications

Phrases like “it must be intuitive” or “it must work well” are useless unless translated into verifiable criteria. If it cannot be tested, it is incomplete.

Tasks that are too large

A task that says “implement the whole flow” usually ends in a hard-to-review PR and traceability debt.

Late ADRs

Writing decisions after the context is forgotten destroys their value. An ADR is useful when it preserves the real discussion.

Acceptance rewritten just to pass

If acceptance criteria are adjusted to match code results, they stop being a contract and become a justification.

Empty traces

It is not enough to say “it is in the PR.” If you cannot follow the thread back to the original requirement, traceability does not exist.

Ghost runbooks

A runbook nobody can execute in an emergency is useless. It must be short, clear, and actionable.

Evaluations without conclusion

Measuring something and not stating which decision was made is incomplete work. The evaluation must leave a concrete position.

Practical checklist

Before closing a feature under SDD, it is worth checking this:

- the requirement is written and includes problem, objective, and impact;
- the spec defines verifiable behavior;
- scope excludes what is out;
- relevant decisions are documented;
- tasks are split into small steps;
- acceptance criteria are concrete;
- there is traceability between issue, spec, PR, and tests;
- a runbook exists if the flow affects operations;
- evaluation or validation notes are present;
- there are no contradictions between document, code, and tests;
- the repository can reconstruct context without depending on oral memory.

Maintenance recommendation

To keep this structure from becoming bureaucratic, it helps to sustain three rules:

1. write little, but useful;
2. update at the same pace as code;
3. close each feature with evidence.

It is not about documenting for the sake of documenting. It is about making documentation work as operational infrastructure. In an AI-enabled repo, that infrastructure is part of the product. It reduces errors, improves review, and allows knowledge to survive context rotation.

Closing

SDD is not a naming trend or a magic replacement for engineering. It is a discipline to organize intention before code obscures it. In repositories where AI participates in writing, review, or analysis, this discipline matters even more because the cost of ambiguity multiplies.

The [.github/orquestador/sdd](https://github.com/orquestador/sdd) structure proposes something simple: separate the right pieces so each one fulfills its function and all together form a readable system. Requirements, specs, decisions, tasks, acceptance

criteria, traces, runbooks, ADRs, and evaluation notes are not decorative folders. They are the mechanism for an idea to move from problem to implementation without losing context along the way.

If the repository adopts this way of working, each feature stops being a leap of faith. It becomes a traceable, reviewable, and improvable process. And that, in an AI-assisted stack, is worth more than any improvised shortcut.

VOLUME 04

AGENTS.md, .github and Prompt-Style Commands

How to build a repository prepared for agents, SDD, Copilot, workflows, and reusable prompts

Why this volume exists

A modern repository should not depend on a human remembering every rule each time they open an AI assistant. Project rules must live in files. Some rules are for humans, others for agents, others for Copilot, others for CI, and others for the SDD process. When these layers are mixed, AI works with incomplete context and the team ends up fixing the same mistakes again and again.

This volume proposes a practical structure for repositories that want to use AI seriously:

1. `AGENTS.md` as the operational contract for code agents.
2. `.github/copilot-instructions.md` as general instructions for Copilot.
3. `.github/instructions/*.instructions.md` as path-specific instructions.
4. `.github/prompts/*.prompt.md` as reusable prompt-style commands.
5. `.github/orquestador/sdd/*` as a system for specifications, decisions, and traceability.
6. `.github/workflows/*` as the only executable automation layer.
7. `.github/orquestador/pipelines/catalog.md` as a governance catalog, not as a second automation engine.

The idea is not to fill the repo with ceremony. The idea is that each file has a clear purpose and that an agent can answer three questions quickly: what project is this, how do we work here, and how do we verify that the change is correct.

Responsibility map

File or folder	Main audience	Responsibility
<code>README.md</code>	New people	Explain product, installation, and usage
<code>AGENTS.md</code>	Code agents	Operational rules, commands, permissions, style, and completion
<code>.github/copilot-instructions.md</code>	GitHub Copilot	Persistent general repository instructions
<code>.github/instructions/*.instructions.md</code>	Copilot by area	Rules applied to specific paths
<code>.github/prompts/*.prompt.md</code>	Team and assistants	Invocable prompts for repeated tasks
<code>.github/orquestador/context/*</code>	Team and agents	Stable context: product, architecture, glossary
<code>.github/orquestador/sdd/*</code>	Team and agents	Specs, decisions, tasks, traceability, runbooks
<code>.github/orquestador/pipelines/catalog.md</code>	Maintainers	Inventory of workflows, permissions, risks, and owners
<code>.github/workflows/*</code>	GitHub Actions	Real automation: CI, reviewers, validations

The golden rule: a context file must not pretend to execute things. A workflow does execute. A catalog documents. A prompt guides. An `AGENTS.md` governs agent behavior. Keeping this separation avoids confusing systems.

Recommended structure

A reasonable base for a repo that wants to work with AI, SDD, and GitHub-first:

```
repo/  
  AGENTS.md  
  README.md  
  src/  
  tests/  
  docs/  
  .github/  
    copilot-instructions.md  
  instructions/  
    frontend.instructions.md  
    backend.instructions.md  
    tests.instructions.md  
  prompts/  
    plan-feature.prompt.md  
    write-spec.prompt.md  
    review-pr.prompt.md  
    generate-tests.prompt.md  
    write-adr.prompt.md  
    qa-harness.prompt.md  
  workflows/  
    ci.yml  
    pr-reviewer.yml  
  orquestador/  
    README.md  
    context/  
      product.md  
      architecture.md  
      glossary.md  
      constraints.md  
    sdd/  
      README.md  
      requirements/  
      specs/  
      decisions/  
      tasks/  
      traces/  
      evals/  
      runbooks/  
    pipelines/  
      catalog.md  
    policies/  
      permissions.md  
      safety.md
```

Not all projects need everything from day one. But it helps for the mental architecture to exist. An MVP can start with `AGENTS.md`, `context/product.md`, `sdd/specs/`, `prompts/`, and `workflows/ci.yml`. The rest is added when real repetition appears.

How to write a good **AGENTS.md**

AGENTS.md is the file that tells an agent how to operate in the repository. It is not a landing page. It is not a product vision. It is not a long manual for humans. It is an operational guide.

It must answer:

1. What the stack is.
2. Where each important thing is.
3. Which commands are used to install, test, validate, and run.
4. Which files or folders are sensitive.
5. What style of changes is expected.
6. Which permissions or actions are forbidden.
7. How to close a task.

Base example:

```

# AGENTS.md

## Project Snapshot

Este repo contiene una app Node.js + TypeScript con API en `src/server`,
frontend en `src/web` y tests en `tests`.

## Commands

- Instalar: `npm ci`
- Desarrollo: `npm run dev`
- Tests: `npm test`
- Typecheck: `npm run typecheck`
- Build: `npm run build`

## Working Rules

- Mantener cambios acotados al pedido.
- No modificar migraciones antiguas sin pedir permiso.
- No tocar secretos ni archivos `.env`.
- Preferir tests cerca del comportamiento cambiado.
- Si un comando falla, reportar el error exacto y el proximo paso.

## SDD

- Specs: `.github/orquestador/sdd/specs/`
- Decisiones: `.github/orquestador/sdd/decisions/`
- Tareas: `.github/orquestador/sdd/tasks/`
- Trazabilidad: `.github/orquestador/sdd/traces/`

## Completion

Antes de cerrar, informar:
- Archivos modificados.
- Pruebas ejecutadas.
- Pruebas no ejecutadas y por que.
- Riesgos residuales.

```

The common mistake is writing an overly philosophical `AGENTS.md`. An agent does not need phrases like "use best practices". It needs commands, paths, limits, and criteria.

Instruction hierarchy

Instructions have precedence. An explicit user request has more weight than a general repo rule. A closer `AGENTS.md` in a subfolder can specialize rules from the root `AGENTS.md`. System or platform instructions have priority over everything else.

The practical way to think about it:

```
System / platform
> User in the conversation
  > AGENTS.md closest to the touched file
    > Root AGENTS.md
      > Supporting documentation
```

This is especially useful in monorepos. A root `AGENTS.md` can describe how work happens globally, while `packages/mobile/AGENTS.md` defines mobile-specific commands and conventions.

`.github/copilot-instructions.md`

GitHub documents custom repository instructions for Copilot as a `.github/copilot-instructions.md` file. Its function is to provide persistent context to Copilot when it works inside the repository.

This file should be shorter than `AGENTS.md`. It does not need to repeat every command. It can focus on style, architecture, response preferences, and expected validations.

Example:

```
# Copilot Instructions

Este proyecto usa TypeScript estricto. Evitar `any` salvo justificacion.
Preferir funciones puras en la capa de dominio.
No crear dependencias nuevas sin explicar el motivo.
Cuando sugieras codigo, incluir pruebas relevantes.
Si falta contexto de negocio, preguntar o marcar supuesto.
```

Use it to guide general quality. Do not turn it into a huge document. If Copilot receives too many general rules, the probability of conflicts or ignored content increases.

`.github/instructions/*.instructions.md`

Path-based instructions let you say: "when you work in this area of the repo, apply these rules." GitHub documents the `NAME.instructions.md` pattern inside `.github/instructions`, with `applyTo` frontmatter.

Backend example:

```
---  
applyTo: "src/server/**/*.ts,tests/server/**/*.ts"  
---  
  
# Backend Instructions  
  
- Validar entradas en la frontera HTTP.  
- Mantener reglas de negocio fuera de handlers.  
- No acceder a la base de datos desde controllers.  
- Agregar tests de casos borde para errores 4xx y 5xx.
```

Frontend example:

```
---  
applyTo: "src/web/**/*.tsx,src/web/**/*.css"  
---  
  
# Frontend Instructions  
  
- Mantener componentes accesibles por teclado.  
- Evitar texto que se desborde en mobile.  
- Usar componentes existentes antes de crear nuevos.  
- No introducir cambios visuales globales sin justificar.
```

The advantage is precision. Instead of letting a frontend rule contaminate backend work, each area gets what it needs.

`.github/prompts/*.prompt.md`

Prompt files are reusable commands. GitHub documents them as Markdown files with a `.prompt.md` extension, usually inside `.github/prompts`. They are designed for tasks the team repeats: planning a feature, reviewing a PR, generating tests, writing an ADR, documenting an API, or preparing onboarding.

Treat them as versioned commands. If a prompt improves, review it in a PR. If it fails, adjust it. If it stops being useful, remove it.

Example `plan-feature.prompt.md`:

```
---
description: "Convertir una idea de producto en plan SDD por slices"
---
```

Usa el contexto de:

- [producto](../orquestador/context/product.md)
- [arquitectura](../orquestador/context/architecture.md)

Entrada del usuario:

`${input:feature:Describe la feature}`

Producir:

1. Problema y objetivo.
2. No objetivos.
3. Supuestos.
4. Requisitos funcionales.
5. Criterios de aceptacion.
6. Slices de implementacion.
7. Tests recomendados.
8. Riesgos y preguntas abiertas.

No inventes reglas de negocio. Marca lo desconocido.

Example `review-pr.prompt.md`:

```
---
description: "Revisar un PR con foco en bugs, riesgos y pruebas"
---
```

Revisa los cambios del PR actual.

Prioridad:

1. Bugs o regresiones.
2. Riesgos de seguridad o permisos.
3. Falta de tests para comportamiento nuevo.
4. Inconsistencias con SDD o ADRs.

Salida:

- Findings con archivo y línea cuando sea posible.
- Preguntas abiertas.
- Resumen breve.

No hagas comentarios de estilo si no afectan mantenimiento o comportamiento.

Example `qa-harness.prompt.md`:

```
---  
description: "Diseñar un harness de evaluacion para una capacidad de IA"  
---
```

```
Capacidad a evaluar:  
${input:capability:Describe la capacidad}
```

```
Disena:  
1. Fixtures de entrada.  
2. Salidas esperadas o criterios.  
3. Rubrica de scoring.  
4. Casos negativos.  
5. Script CLI minimo.  
6. Como integrarlo en CI.  
7. Riesgos de falsos positivos.
```

These files make team knowledge invocable. They do not replace human judgment, but they reduce variability.

.github/orquestador

The **.github/orquestador** folder acts as the home of the working system. The name is not magical. What matters is that it has a clear responsibility: gathering context, SDD, policies, and catalogs that guide humans and agents.

A useful convention:

.github/orquestador/	
README.md	Index of the repo operating system
context/	Stable context
sdd/	Specifications and traceability
prompts/	Internal prompts if <code>`github/prompts`</code> is not used
pipelines/	Catalog of workflows and automations
policies/	Permissions, security, risk criteria

If the team uses GitHub Copilot heavily, it is best to keep **.github/prompts** for tool compatibility and leave **.github/orquestador** for context and SDD. If another agent is used, **orquestador/prompts** can also exist. The important thing is not to duplicate without need.

SDD inside the repo

SDD means working from specifications, not impulses. In an AI-enabled repository, SDD serves a critical function: it prevents the agent from implementing a creative interpretation of the request.

A minimal spec should include:

1. Problem.

2. Objective.
3. Non-objectives.
4. Requirements.
5. Acceptance criteria.
6. Edge cases.
7. Technical impact.
8. Slice-based plan.
9. Tests.
10. Traceability with issue, PR, and decisions.

Example path:

```
.github/orquestador/sdd/specs/2026-05-08-reasignar-reclamo.md
```

Example header:

```
# Reasignar reclamo a otro equipo

Estado: Draft
Owner: Producto / Backend
Issue: #123
PRs: pendiente

## Problema

Los operadores no pueden mover un reclamo cuando fue derivado al equipo incorrecto.

## Criterios de aceptacion

- Un operador autorizado puede reasignar el reclamo.
- La reasignacion queda auditada.
- El equipo anterior y el nuevo quedan visibles en el historial.
- Si el reclamo esta cerrado, no puede reasignarse.
```

When the agent implements, it must be able to read that spec and know what "done" means.

Workflows as the only executable layer

If the repository uses GitHub as the main surface, it is best for `.github/workflows` to be the only executable automation layer. Everything else can document, guide, or govern. This separation avoids two problems: duplicated automations and uncertainty about where execution really happens.

Conservative starter pattern:

```

name: Safe PR Reviewer

on:
  pull_request:
    types: [opened, synchronize, reopened]

permissions:
  contents: read
  pull-requests: read
  issues: write

concurrency:
  group: pr-reviewer-${{ github.event.pull_request.number }}
  cancel-in-progress: true

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - name: Comment with checklist
        uses: actions/github-script@v7
        with:
          script: |
            const body = [
              "Revision automatica inicial:",
              "- Verificar tests.",
              "- Confirmar criterios SDD.",
              "- Revisar permisos y secretos.",
              "- Mantener cambios acotados."
            ].join("\n");
            await github.rest.issues.createComment({
              owner: context.repo.owner,
              repo: context.repo.repo,
              issue_number: context.payload.pull_request.number,
              body
            });

```

This workflow does not checkout, does not execute PR code, and comments conservatively. It is a good first automation because it adds order without opening a large risk surface.

Pipeline catalog

The catalog does not execute. It documents. It should answer:

1. Which workflow exists.
2. Which event triggers it.

3. Which permissions it uses.
4. Which risks it has.
5. Who maintains it.
6. Which output it produces.
7. What it is not authorized to do.

Example:

```
# Pipeline Catalog

## safe-pr-reviewer

- Archivo: `.github/workflows/pr-reviewer.yml`
- Evento: `pull_request`
- Permisos: `contents: read`, `pull-requests: read`, `issues: write`
- Ejecuta código del PR: no
- Output: comentario con checklist
- Owner: Platform
- Riesgo principal: ruido en PRs si las reglas no se mantienen
- Estado: activo
```

When someone asks "what automations does this repo have?", the catalog gives the answer. When someone asks "what gets executed?", the answer is still `.github/workflows`.

How the pieces connect

A complete flow could be:

1. A human creates an issue.
2. They run `plan-feature.prompt.md` to convert the idea into a spec.
3. They save the spec in `.github/orquestador/sdd/specs/`.
4. The agent reads `AGENTS.md`, context, and spec.
5. The agent implements a scoped slice.
6. It runs the tests indicated by `AGENTS.md`.
7. It opens a PR.
8. `pr-reviewer.yml` comments a checklist.
9. Another agent or a human uses `review-pr.prompt.md`.
10. Traceability is updated in SDD.

What is valuable is that each step leaves traces. The team does not depend on remembering what was discussed in a chat.

Common errors

Error	Consequence	Correction
Huge <code>AGENTS.md</code>	The agent ignores parts	Keep it operational and link long docs
Repeating rules in five files	Conflicting instructions	Define an owner per layer
Unversioned prompts	Each person uses variants	Store them in <code>.github/prompts</code>
Workflows with broad permissions	Unnecessary risk	Minimum permissions per workflow
Catalog that promises execution	Operational confusion	Catalog documents, workflows execute
Specs without acceptance criteria	Ambiguous implementations	Close each spec with tests and examples

Bootstrap checklist

To leave a repo ready:

1. Create `AGENTS.md` with real commands.
2. Create a short `.github/copilot-instructions.md`.
3. Create `.github/instructions/` only if there are path-based rules.
4. Create `.github/prompts/` with 3 to 5 useful prompts.
5. Create `.github/orquestador/context/product.md`.
6. Create `.github/orquestador/context/architecture.md`.
7. Create `.github/orquestador/sdd/README.md`.
8. Create `specs/`, `decisions/`, `tasks/`, `traces/`, `evals/`, `runbooks/`.
9. Create `.github/orquestador/pipelines/catalog.md`.
10. Verify that `.github/workflows` is the only layer that executes automation.
11. Make a test PR to confirm the instructions are understandable.

Verified sources

GitHub documents custom repository instructions for Copilot, including `.github/copilot-instructions.md`, `.github/instructions/*.instructions.md`, and the use of `AGENTS.md` for agent instructions:

- [GitHub Docs - Adding repository custom instructions for GitHub Copilot](#)
- [GitHub Docs - Prompt files](#)
- [GitHub Docs - Your first prompt file](#)
- [openai/agents.md](#)

Note: Copilot prompt files were documented as public preview when these sources were verified on 2026-05-08, so it is worth checking the official documentation before imposing them as a rigid enterprise standard.

Closing

The goal is not to have an impressive `.github` folder. The goal is that every agent entering the repo can work with less guessing, more verification, and better memory. `AGENTS.md` gives operating rules. SDD provides the product and technical contract. Prompt files turn repeated tasks into commands. Workflows run validations. The catalog explains the system.

When those pieces are aligned, AI stops being a loose chat and starts being work infrastructure.

VOLUME 05

Prompt Engineering and Harness Engineering

From loose prompts to versioned, evaluable, and production-grade systems

The difference between a useful experiment and a reliable system is not just writing a good prompt. An isolated prompt can solve a specific task, but a serious product needs more: versions, test cases, evaluation criteria, observability, and security rules. That set is what turns an idea into an operable capability.

This volume starts from a simple idea: a prompt should not live as a loose string pasted into an app, a notebook, or a comment. If an instruction matters to the business, it must be auditable, comparable, testable, and deployable with discipline. That is where harness engineering comes in: the design of the environment that executes, measures, and controls that prompt.

The practice shifts the focus. It is no longer about asking "what do I tell the model?", but "how do I make sure this task always runs with the same criteria, can be verified, and does not break as the system grows?". That is the transition from handcrafted prompt engineering to productive prompt engineering.

What changes when the prompt goes into production

In a demo, a prompt can be the exact text that gave you a good result today. In production, that same text stops being enough because conditions appear that did not matter before:

1. The model changes.
2. The temperature changes.
3. The user context changes.
4. Input data changes.
5. The team that maintains the system changes.
6. Legal, security, or brand requirements appear.

When that happens, the problem is not only "response quality". The problem is control. You need to know which instruction was used, with which version, on which inputs, with which tools, and whether behavior remains within expectations.

That is why a production prompt system usually includes these parts:

- persistent instructions, which describe identity and stable behavior;
- task prompts, which solve a specific request;
- fixtures, which represent well-defined input cases;
- golden tests, which set expected outputs or observable properties;
- evals, which measure quality with a rubric;
- regression tests, which detect degradations;
- permission and security rules;
- logging and observability to review results in context.

These are not decorative layers. They are the minimum infrastructure required to trust an assistant, a classifier, a writer, a router, or an agent.

Anatomy of a good prompt

A good prompt is not long or short by definition. It is useful when it reduces ambiguity, orders priorities, and makes clear what to do when data is missing. In practice, the best prompts share a similar structure.

1. Explicit objective

The model must know what problem it is solving. "Help the user" is not enough. It is better to specify the objective in operational terms.

Example:

```
Your task is to transform informal requests into clear, actionable, and complete work tickets.
```

2. Operating context

The prompt must say in which environment it is used and what limits it has. Writing for support is not the same as writing for a sales agent or an internal classifier.

Example:

```
This assistant is used by an operations team. Prioritize clarity, traceability, and professional language.
```

3. Quality criteria

If you do not define quality, the model invents it. A good prompt marks what is valued: precision, concision, coverage, tone, safety, format.

Example:

```
The response must be concrete, must not invent data, and must separate observable facts from assumptions.
```

4. Constraints

Constraints prevent comfortable but useless answers. This includes format, length, available tools, languages, exclusions, and security rules.

Example:

```
Do not use tables if the information is uncertain. Do not assume permissions. Do not execute actions without confirmation.
```

5. Uncertainty policy

A mature prompt indicates what to do when context is missing. That reduces hallucinations and shaky responses.

Example:

```
If critical data is missing, ask one clarification question. If the data is not critical, proceed with an explicit assumption.
```

6. Output format

Output must be easy to consume by humans or by another system layer. If you need JSON, specify the schema. If you need bullets, say so. If you need a template, define it.

Example:

```
Always return:  
1. Summary  
2. Risks  
3. Next steps
```

7. Examples

Examples are not decoration. They are semantic anchors. They help fix tone, detail level, and boundary decisions. They are especially useful when a task has format or criteria ambiguity.

A good prompt usually combines instructions with 1 or 2 compact examples. No need to overload it: too many examples also introduce noise.

Persistent instructions vs task prompts

A common mistake is mixing everything into one instruction. That makes the system fragile. The useful split is:

- persistent instructions: what should not change between tasks;
- task prompts: what changes for each request;
- dynamic context: user data, files, session state, tools, temporary memory.

Persistent instructions

This is the identity and policy layer. It defines role, tone, criteria priority, safety limits, language, and expected behavior.

Example:

```
You are an operations assistant. You answer in neutral Rioplatense Spanish, with a clear and serious style. You prioritize precision, safety, and actionable steps.
```

Task prompts

These are point instructions for a specific execution. They should be brief, specific, and focused on the result.

Example:

Using this text, write an executive summary of at most 180 words and end with 3 concrete risks.

Practical rule

If an instruction repeats in every case, it likely belongs in the persistent layer. If it changes every execution, it belongs in the task layer. If it lives in user input, do not mix it with policy. Separating these things reduces errors and makes the system maintainable.

Fixtures: concrete cases for testing behavior

A fixture is a controlled input case that represents a realistic situation. In prompt engineering, fixtures are essential because they let you verify whether the system responds well to typical, rare, or dangerous scenarios.

One happy-path example is not enough. A serious harness needs fixtures that cover:

- clean inputs;
- incomplete inputs;
- contradictory inputs;
- noisy inputs;
- ambiguous requests;
- attempts to force an unsafe output;
- format edge cases.

Fixture example

```
{
  "id": "ticket-001",
  "input": "I need someone to review last month's invoice because I think it came wrong.",
  "expected_traits": [
    "asks for clarification if identifier is missing",
    "does not invent data",
    "keeps professional tone",
    "proposes next step"
  ]
}
```

What makes a fixture good

A good fixture does not try to "beat" the model with tricks. It describes a useful situation. It should be:

- stable;
- reproducible;
- readable;

- representative;
- easy to extend.

If the fixture changes all the time, it is useless for comparing versions. If it is too artificial, it does not reflect real usage. Quality lies in the balance.

Golden tests: lock expected behavior

Golden tests are checks against an expected output or very concrete output properties. They help detect regressions when the prompt, model, or tool chain changes.

There are two common forms:

Exact golden

Useful when output must be very stable, for example a structured format.

Example:

```
{
  "id": "routing-01",
  "expected": {
    "category": "billing",
    "confidence": "high"
  }
}
```

Property-based golden

Useful when you do not want to freeze the full text, but you do want to freeze behavior.

Example:

- response must not invent data;
- it must include a warning;
- it must return exactly 3 steps;
- it must use the expected language;
- it must not mention unauthorized tools.

This second format is more flexible and is often more useful for natural-language systems. Gold is not always the exact string; often it is compliance with observable rules.

Harness CLI: run, compare, repeat

The harness is the environment that runs prompts with fixtures, logs results, and compares them against a reference. A well-designed CLI makes all of that repeatable from terminal and CI.

A reasonable structure separates:

- prompt definition;
- fixture definition;
- model configuration;
- batch execution;
- evaluation;
- report;
- result export.

Possible structure

```
prompt-harness/  
  prompts/  
    system.md  
    task.md  
  fixtures/  
    inbox.jsonl  
    safety.jsonl  
    formatting.jsonl  
  evals/  
    rubric.md  
    scoring.ts  
  runs/  
    2026-05-08T10-30-00Z.jsonl  
  reports/  
    latest.md  
  src/  
    cli.ts  
    harness.ts  
    loader.ts  
    evaluator.ts
```

Example commands

```
node src/cli.js run --prompt prompts/system.md --fixtures fixtures/inbox.jsonl  
node src/cli.js eval --run runs/latest.jsonl --rubric evals/rubric.md  
node src/cli.js compare --baseline runs/baseline.jsonl --candidate runs/latest.jsonl  
node src/cli.js report --input runs/latest.jsonl --output reports/latest.md
```

What the CLI should do

A useful CLI does not only call the model. It also:

- validates that files exist;

- normalizes inputs;
- logs prompt and model version;
- stores timestamps;
- serializes raw responses;
- computes metrics;
- emits a clear summary for CI.

If the harness only prints pretty text, it is a demo. If it also leaves traceability and comparison, it starts to become infrastructure.

Evals and rubrics: measure beyond "I like it"

Prompt evaluation cannot depend only on intuition. You need repeatable criteria. That is where the rubric comes in: an explicit definition of what "good" means.

Simple rubric

A rubric can score dimensions such as:

- context fidelity;
- completeness;
- accuracy;
- format;
- tone;
- safety;
- actionable usefulness.

Example:

```
Score 0-2 per dimension:  
0 = critical failure  
1 = partial  
2 = meets criteria
```

Evaluation example

Dimension	Criterion
Fidelity	Does not invent data or contradict the input
Format	Follows the requested structure
Tone	Keeps professional language
Actionability	Provides useful next steps
Safety	Does not execute or recommend forbidden actions

Automatic and human evaluation

It is best to combine both approaches.

- automatic: for objective rules, format, length, field presence, forbidden patterns;
- human: for quality nuances, clarity, persuasion, usefulness, and alignment.

In real systems, the most common mistake is using weak automatic metrics as if they solved everything. They help, but they do not replace critical reading. Good practice is using automation for filtering and a human rubric for decisions.

Regression tests: do not break what already worked

A regression test compares behavior across versions. The goal is not to freeze the system forever, but to detect undesired changes.

In a mature flow, every prompt or model change should answer:

1. what improved;
2. what got worse;
3. what new cases appeared;
4. what is accepted as a trade-off;
5. what needs rollback.

Typical regression cases

- the new prompt becomes more verbose than needed;
- a safety warning disappears;
- the model stops asking for clarification;
- language changes;
- JSON format breaks a parser;
- a tool is invoked when it should not be.

Practical strategy

Keep a small set of critical fixtures and a broader set of exploratory fixtures. Critical ones protect the essentials. Exploratory ones show how the system behaves outside the main path.

If a regression test fails, it is not enough to "fix output". You need to understand whether the problem is in:

- the prompt;
- the model;
- post-processing;
- configuration;
- policy;
- data set.

Security and permissions

When a model produces actions, good answers are not enough. It also has to operate within clear limits. In tool-enabled systems, security is part of prompt and harness design.

Basic principles

- least privilege;
- confirmation for sensitive actions;
- separation between suggesting and executing;
- input and output validation;
- decision logging;
- explicit blocking of dangerous actions.

Rule example

```
If an action modifies data, charges money, deletes information, or sends external messages, request human confirmation before executing it.
```

Secure prompt

The prompt should not assume permissions or credentials. It also should not encourage the model to "solve on its own" things that require external validation.

Example:

```
Do not assume you have access to external systems. If an impactful action is needed, describe it and request confirmation.
```

Secure harness

The harness should be able to simulate permissions and test limits:

- without internet access;
- with tools disabled;
- with fake credentials;
- in read-only mode;
- with confirmation required.

That lets you verify the system does not only work when everything is enabled, but also when it operates under restrictions.

Observability: see what actually happened

Observability is what lets you debug and learn from the system once it leaves the lab. If a prompt fails in production, you need to reconstruct context.

What to log

- prompt version;
- model version;
- date and time;
- summarized input;
- raw output;
- tools used;
- latency;
- estimated cost;
- errors;
- policy decision;
- fixture or case identifier.

Log example

```
{
  "run_id": "2026-05-08T10:30:00Z",
  "prompt_version": "1.4.2",
  "model": "gpt-5.3",
  "fixture_id": "safety-03",
  "latency_ms": 1840,
  "tools_used": ["search"],
  "outcome": "needs_review"
}
```

What to inspect first

When something fails, review:

1. original input;
2. actual prompt applied;
3. model configuration;
4. available tools;
5. raw output;
6. post-processing;
7. evaluation criteria.

Without observability, every bug looks like magic. With observability, it becomes a sequence of auditable decisions.

A minimal harness example

To make the idea concrete, imagine a system that classifies internal requests and responds with a short plan. The minimum architecture can look like this:

```
1. load persistent instructions
2. load task prompt
3. load fixture
4. build final message
5. run model
6. validate format
7. score with rubric
8. save result
9. compare against baseline
10. report
```

Pseudocode

```
const system = loadFile("prompts/system.md")
const task = loadFile("prompts/task.md")
const fixtures = loadJsonl("fixtures/inbox.jsonl")

for (const fixture of fixtures) {
  const input = buildInput(system, task, fixture)
  const output = await model.run(input)
  const score = evaluate(output, fixture.expected_traits)
  saveRun({ fixtureId: fixture.id, output, score })
}
```

What matters in the example

You do not need sophistication to start. The essential part is that the flow is:

- explicit;
- reproducible;
- versioned;
- evaluable;
- comparable.

Once that exists, you can scale. Before that, everything is hard to maintain.

Criteria for iterating prompts without breaking the system

When you improve a prompt, do not edit blindly. Better to follow a short, disciplined cycle.

Iteration checklist

- define the exact problem;
- choose a representative fixture;
- write the improvement hypothesis;
- change only one thing at a time;
- run the harness;
- compare against baseline;
- review new failures;
- accept the change or revert it;
- document the decision.

Practical rules

1. Do not mix format improvements with policy changes in the same iteration.
2. Do not use one happy example to justify a change.
3. Do not declare victory without checking regressions.
4. Do not save a new prompt without its version and rationale.
5. Do not put critical business logic only inside the prompt if it can live better in code.

When to move logic outside the prompt

A prompt should not carry everything. If a rule is strict, verifiable, and central to the business, it is often better to encode it outside the model.

Examples:

- schema validation;
- permission rules;

- deterministic routing;
- sensitive data filtering;
- format normalization;
- metric computation.

The prompt stays for what it does best: interpret, prioritize, write, synthesize, and decide with context. Code stays for what it does best: validate, control, and execute deterministically.

Maturity signals

You know you moved from a "loose prompt" to a "system" when you can answer these questions without improvising:

- what prompt version is in production?
- with which fixtures is it validated?
- what criteria does the rubric define?
- what changed against baseline?
- what actions are allowed?
- what was recorded in logs?
- which regressions are tolerated and which are not?

If those answers are scattered, the system still depends too much on human memory.

Operational summary

Prompt engineering is not only writing better. It is designing an instruction interface that can be sustained over time. Harness engineering is the support that makes that sustainability possible: it loads fixtures, runs versions, evaluates results, records evidence, and protects the system from accidental changes.

Practical discipline fits in a simple formula:

- separate persistent instructions from task prompts;
- use real and representative fixtures;
- set expectations with golden tests and rubrics;
- run regression tests before publishing changes;
- control permissions and sensitive actions;
- observe what happens in production;
- version everything that matters.

When this exists, the prompt stops being a bet. It becomes an engineering component.

Final checklist

- ☐ Define the system's persistent instruction.
- ☐ Separate the task prompt from dynamic context.
- ☐ Create fixtures for happy, ambiguous, and risky cases.
- ☐ Write at least one golden test per critical behavior.
- ☐ Design a simple, repeatable rubric.
- ☐ Implement a harness CLI with `run`, `eval`, and `report`.
- ☐ Save versioning for prompts, model, and configuration.
- ☐ Log records and latency per execution.
- ☐ Test permissions, fallbacks, and no-tool cases.
- ☐ Compare every change against a baseline.
- ☐ Document change and rollback decisions.

That set is enough to move from a promising idea to a productive, auditable, and maintainable capability.